```
$ ./leitor-exibidor -s metodos examples/tableswitch.class
```

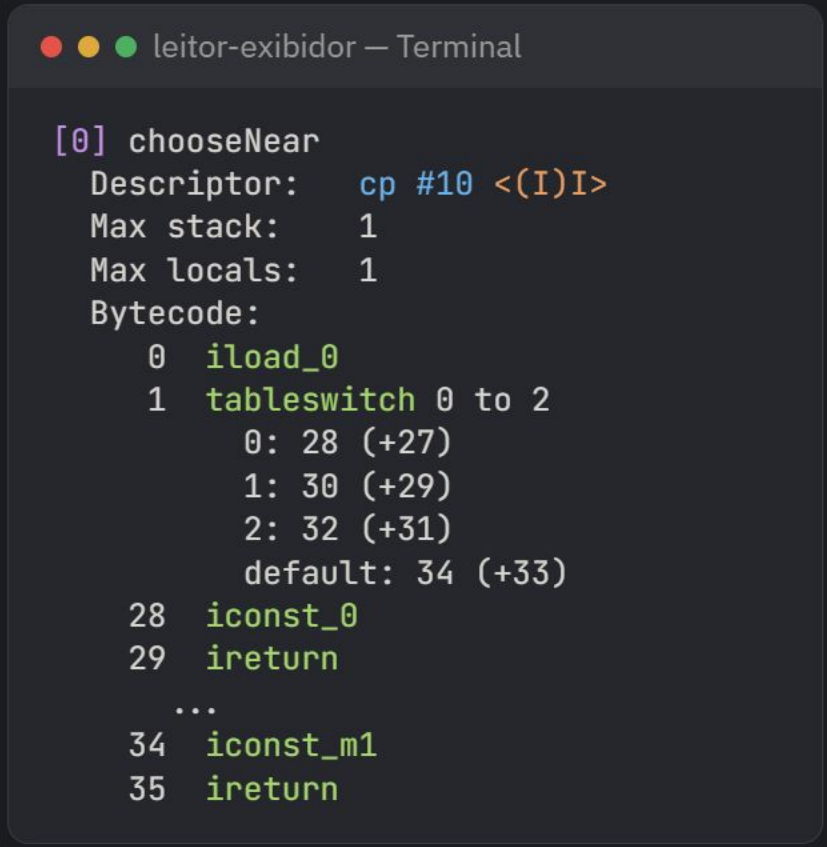
# Leitor-exibidor de arquivos .class

Software Básico, UnB

C99 · GCC · formato inspirado no jclasslib

Antonio Coelho · Gabriel Pinto · Leonardo Mileo · Lucas Mendes

github.com/LucasMABF/SB



USO

# Como compilar e executar

## Compilar (GCC, C99)

```
make
# ou, sem o Make:
gcc -std=c99 -Wall -Wextra -pedantic \
  -O2 -g src/*.c -o leitor-exibidor
```

Opções: **-std=c99 -Wall -Wextra -pedantic -O2 -g**. Nenhum warning no GCC 6.3 (MinGW) nem no GCC 16 (MinGW-w64).

No Windows: **mingw32-make**, e o executável é **leitor-exibidor.exe**.

## Executar

```
./leitor-exibidor X.class
./leitor-exibidor -s pool X.class
./leitor-exibidor -o saida.txt X.class
./leitor-exibidor -i X.class
```

**-s** uma seção (geral, pool, interfaces, fields, metodos, atributos) · **-o** grava em arquivo · **-i** menu interativo · **-h** ajuda

**make test** roda em todas as classes de teste e conta as pendências.

CÓDIGO

# Organização do código

| Arquivo | Responsabilidade |
|---|---|
| reader.c | Leitura big-endian com checagem de limites |
| classfile.c | Estruturas do .class e parser (JVMS §4) |
| constant_pool.c | Resolução e exibição do pool, Modified UTF-8, float/double |
| opcodes.c · bytecode.c | Tabela de opcodes e disassembler do atributo Code |
| attributes.c · attr_decoders.c | Despacho por nome e decodificadores de cada atributo |
| access_flags.c | Flags por contexto (classe, field, método, classe interna) |
| printer.c · main.c | Seções da saída e opções de linha de comando |

Os atributos ficam como bytes brutos após o parse e só são decodificados na exibição: cada decodificador é independente.

SOMAR.CLASS

# Informações gerais

- Magic number e versões, com o nome da versão do Java (52 → 1.8)
- Access flags em hexadecimal e decodificadas
- This e super class com o índice no pool e o nome resolvido

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s geral examples/Somar.class

======================================================
  GENERAL INFORMATION
======================================================
Magic number:          0xCAFEBABE
Minor version:         0
Major version:         52 [1.8]
Constant pool count:   14
Access flags:          0x0600 [interface abstract]
This class:            cp #1 <Somar>
Super class:           cp #2 <java/lang/Object>
Interfaces count:      0
Fields count:          1
Methods count:         1
Attributes count:      1
```

SOMAR.CLASS

# Pool de constantes

- Cada entrada com o índice [n] e a tag, como no jclasslib
- Referências mostram cp #índice e a constante resolvida entre < >
- Double e Long mostram high/low bytes e ocupam dois slots

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s pool examples/Somar.class

Constant pool count: 14

[  1] CONSTANT_Class_info
      Class name:           cp #12 <Somar>
[  2] CONSTANT_Class_info
      Class name:           cp #13 <java/lang/Object>
[  3] CONSTANT_Utf8_info
      Length of byte array:  2
      Length of string:      2
      String:                pi
  ...
[  6] CONSTANT_Double_info
      High bytes:            0x400921CA
      Low bytes:             0xC083126F
      Double:                3.1415
[  7] (large numeric continued)
```

SOMA_CERTO.CLASS · SOMAR.CLASS

# Interfaces e fields

- Interfaces: índice no pool e nome da interface
- Fields: nome, descritor e flags do contexto de field
- Atributos de cada field, como o ConstantValue de pi

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s interfaces examples/soma_certo.class
Interfaces count: 1
[0] Interface:          cp #4 <Somar>
```

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s fields examples/Somar.class
Fields count: 1
[0] pi
    Name:                   cp #3 <pi>
    Descriptor:             cp #4 <D>
    Access flags:           0x0019 [public static final]
    Attributes count:       1
    [0] ConstantValue
        Attribute length:       2
        Constant value index:   cp #6 <3.1415>
```

TABLESWITCH.CLASS

# Métodos e bytecode

- Atributo Code: max stack, max locals, tamanho do código e o bytecode
- Cada instrução com o pc e o mnemônico
- tableswitch: destino e offset de cada caso; o padding conta a partir do início do código

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s metodos examples/tableswitch.class
[0] chooseNear
    Descriptor:             cp #10 <(I)I>
    ...
    [0] Code
        Max stack:              1
        Max locals:             1
        Code length:            36
        Bytecode:
             0  iload_0
             1  tableswitch 0 to 2
                   0: 28 (+27)
                   1: 30 (+29)
                   2: 32 (+31)
                   default: 34 (+33)
            28  iconst_0
            29  ireturn
              ...
            34  iconst_m1
            35  ireturn
        Exception table length: 0
```

LOOKUPSWITCH.CLASS

# Bytecode: lookupswitch

- Número de pares e cada par chave → destino
- Mesma regra de padding do tableswitch
- Destino absoluto e offset relativo ao pc da instrução

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s metodos examples/lookupswitch.class
[0] chooseFar
    Descriptor:             cp #10 <(I)I>
    ...
        Bytecode:
             0  iload_0
             1  lookupswitch 3
                   -100: 36 (+35)
                   0: 38 (+37)
                   100: 40 (+39)
                   default: 42 (+41)
            36  iconst_m1
            37  ireturn
            38  iconst_0
            39  ireturn
            40  iconst_1
            41  ireturn
            42  sipush 200
            45  ireturn
        Exception table length: 0
```

BELOTE.CLASS · MAIN

# Operandos que apontam para o pool

- new, anewarray, ldc, invokespecial... mostram #índice e o valor
- Methodref resolvido como classe.método : descritor
- Desvios: destino e offset, como if_icmpge 48 (+25) e goto 20 (-25)

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s metodos examples/Belote.class
[4] main  <([Ljava/lang/String;)V>  Bytecode:
     0  new #43 <Belote>
     3  dup
     4  invokespecial #44 <Belote.<init> : ()V>
     7  astore_1
     8  invokestatic #45 <Belote.printStatus : ()V>
    11  iconst_4
    12  anewarray #36 <Carta>
       ...
    20  iload 6
    22  iconst_4
    23  if_icmpge 48 (+25)
    26  aload 5
    28  iload 6
    30  new #36 <Carta>
    33  dup
    34  iconst_1
    35  ldc #42 <"1">
    37  iconst_1
    38  invokespecial #39 <Carta.<init> : (ILjava/lang/String;I)V>
    41  aastore
    42  iinc 6 by 1
    45  goto 20 (-25)
```

RECURSOS.CLASS

# Atributos da classe

- Nome do atributo pelo pool e tamanho em bytes
- Conteúdo decodificado conforme o tipo: Signature, SourceFile, BootstrapMethods, InnerClasses...
- 25 atributos suportados; os desconhecidos aparecem em hexadecimal

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s atributos tests/classes/Recursos.class
Attributes count:      4
[0] Signature
    Signature index:        cp #176 <<T::Ljava/lang/Comparable<TT;>;>…>
[1] SourceFile
    Attribute name index:   cp #177 <SourceFile>
    Attribute length:       2
    Source file name:       cp #178 <Recursos.java>
[2] BootstrapMethods
    Attribute length:       22
    Number of methods:      2
    ...
[3] InnerClasses
    Attribute length:       34
    Number of classes:      4
    ...
    [1]
      Inner class:          cp #196 <Recursos$Aninhada>
      Outer class:          cp #15 <Recursos>
      Inner name:           cp #198 <Aninhada>
      Access flags:         0x0008 [static]
```

CAFEBABE.CLASS

# Arquivo que não é um .class

leitor-exibidor — Terminal

```
$ ./leitor-exibidor examples/cafebabe.class
Erro ao ler 'examples/cafebabe.class': magic number inválido (0x4D5A9000, esperado 0xCAFEBABE): não é um arquivo .class
$ echo $?
1
```

- Os 4 primeiros bytes são 4D 5A 90 00 (“MZ”, o cabeçalho de um executável do Windows), não CA FE BA BE.
- O leitor para na verificação do magic number e sai com código 1; a mensagem vai para stderr.
- Arquivos truncados ou corrompidos também geram uma mensagem clara: mais de 2.000 variações testadas, nenhuma derrubou o programa.

ALÉM DOS EXEMPLOS DA DISCIPLINA

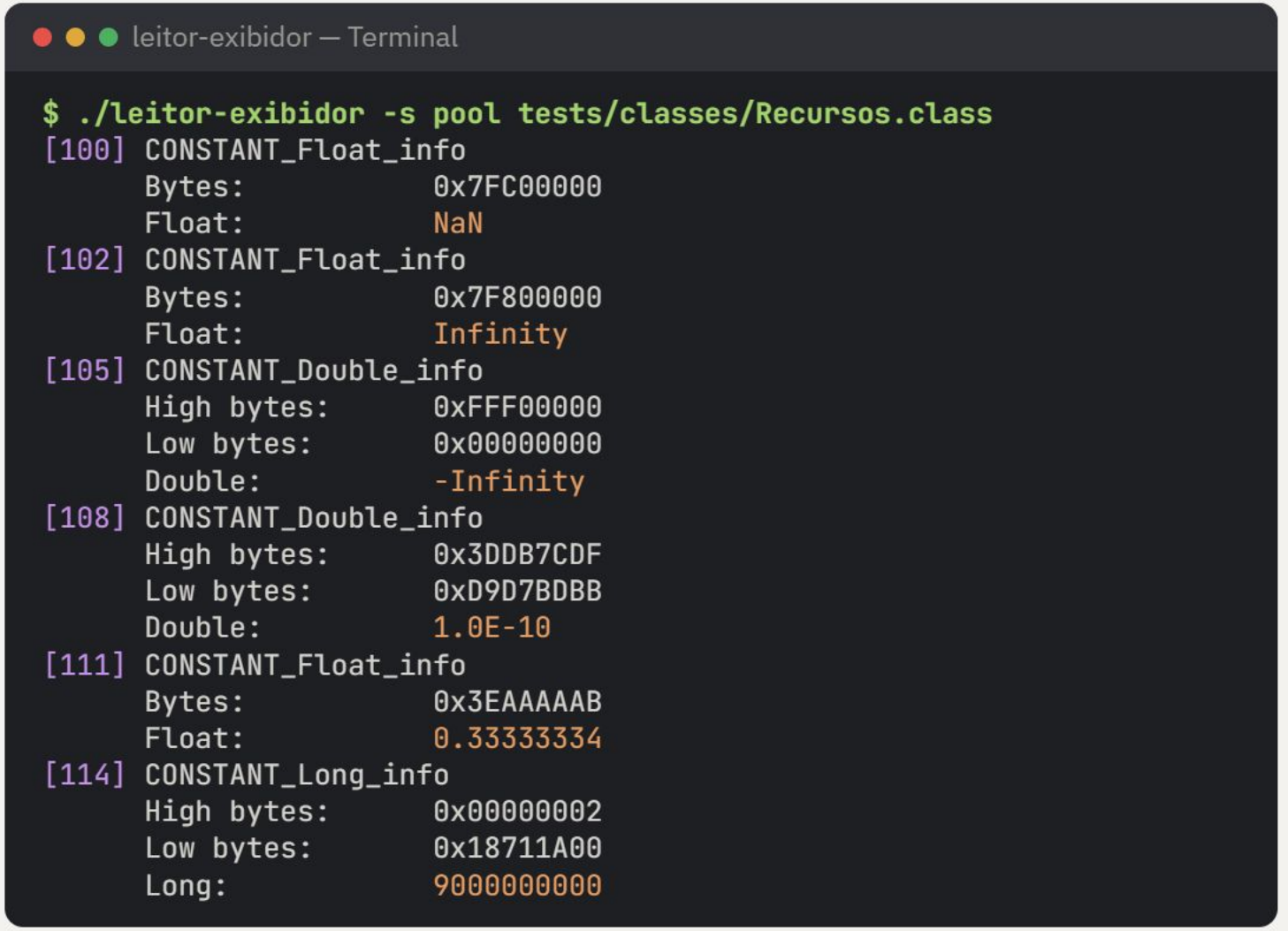
# 30 classes conferidas com javap -v -p: mesmas constantes, mesmas instruções, mesmos destinos de switch.

Classes extras em tests/java, compiladas com javac 21 (--release 8 e 17), cobrem o que os exemplos não cobrem: wide, NaN e Infinity, strings com emoji e \n, lambdas, anotações, records e StackMapTable.

RECURSOS.CLASS

# Constantes numéricas especiais

- NaN, Infinity e -Infinity identificados pelos bits
- Float e Double no formato do Java: 1.0E-10, 0.33333334
- Menor número de dígitos que reproduz o valor exato



RECURSOS.CLASS

# Strings em Modified UTF-8

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s pool tests/classes/Recursos.class

[120] CONSTANT_String_info
      String:                cp #121 <linha1\nlinha2\ttab ação 😀 \0fim>
[121] CONSTANT_Utf8_info
      Length of byte array:  37
      Length of string:      30
      String:                linha1\nlinha2\ttab ação 😀 \0fim
```

- O caractere nulo (C0 80) vira \0 e o emoji (dois surrogates de 3 bytes cada) vira um único caractere.
- \n e \t são escapados para não quebrar o layout: 37 bytes no arquivo e 30 caracteres (code points) na string.

RECURSOS.CLASS · TENTAR() COM TRY / CATCH / FINALLY

# Tabela de exceções e StackMapTable

Cada entrada da tabela aponta para o tipo capturado no pool (cp #0 = any, o finally); cada frame do StackMapTable mostra o tipo e o pc absoluto.

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s metodos tests/classes/Recursos.class
[4] tentar  ...
    Exception table length: 4
      Nr.  Start PC End PC   Handler  Catch type
      0    0        5        17       cp #60 <java/lang/NumberFormatException>
      1    0        5        32       cp #0 <any>
      2    17       20       32       cp #0 <any>
      3    32       34       32       cp #0 <any>
    ...
    [3] StackMapTable
        Number of entries:     2
        [0] same_locals_1_stack_item_frame (frame_type 81), offset_delta 17 → pc 17
            Stack: [cp #60 <java/lang/NumberFormatException>]
        [1] same_locals_1_stack_item_frame (frame_type 78), offset_delta 14 → pc 32
            Stack: [cp #161 <java/lang/Throwable>]
```

LARGO.CLASS

# Instrução wide

- Método com 300 variáveis locais
- A partir do índice 256 o javac usa wide istore / wide iload
- wide iinc 299 by 1000: índice e constante de 16 bits

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s metodos tests/classes/Largo.class
    ...
    1130  istore 253
    1132  sipush 254
    1135  istore 254
    1137  sipush 255
    1140  istore 255
    1142  sipush 256
    1145  wide istore 256
    1149  sipush 257
    1152  wide istore 257
    ...
    1443  sipush 299
    1446  wide istore 299
    1450  wide iinc 299 by 1000
    1456  iload_0
    1457  wide iload 299
    1461  iadd
    1462  ireturn
```

MODERNO$PONTO.CLASS · COMPILADA COM --RELEASE 17

# Anotações, NestHost e Record

Atributos do Java 11+: anotação com todos os tipos de valor, NestHost e os componentes do record.

leitor-exibidor — Terminal

```
$ ./leitor-exibidor -s atributos 'tests/classes/Moderno$Ponto.class'
[1] RuntimeVisibleAnnotations
    Number of annotations: 1
      [0] @LModerno$Info;(autor="SB", versao=2, cor=LModerno$Cor;.VERDE, tipo=class Ljava/lang/String;, tags=
{"a", "b"})
[2] NestHost
    Host class:             cp #32 <Moderno>
[3] Record
    Components count:       2
    [0] x
      Name:                 cp #11 <x>
      Descriptor:           cp #12 <I>
    [1] y
      Name:                 cp #15 <y>
      Descriptor:           cp #12 <I>
```

MODERNO$PONTO.CLASS · INVOKEDYNAMIC

# BootstrapMethods e MethodHandle

Cada bootstrap method é um MethodHandle resolvido (tipo de referência, classe, método e descritor), seguido dos argumentos estáticos com os valores do pool.

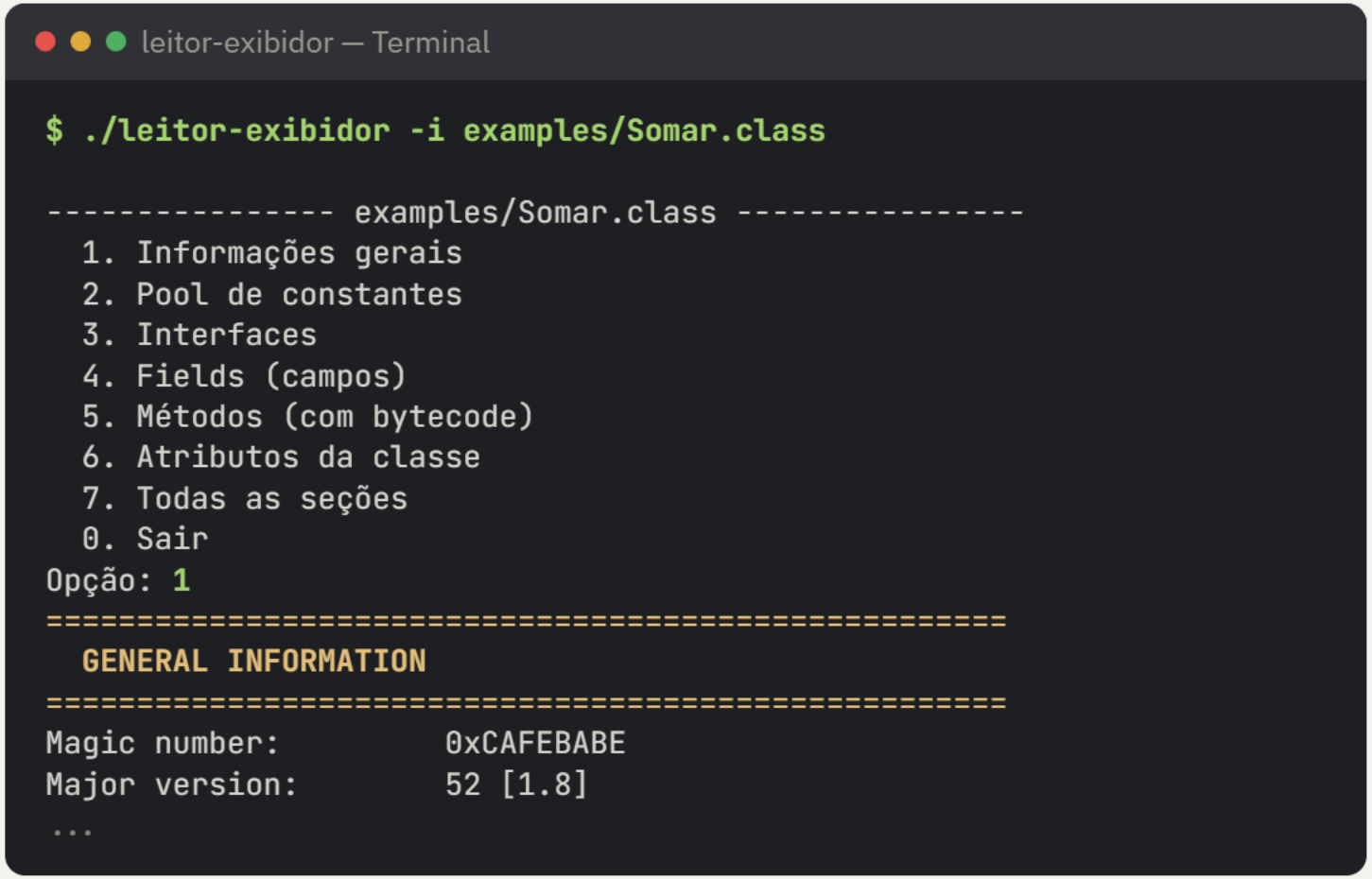
leitor-exibidor — Terminal (descritores longos abreviados com …)

```
$ ./leitor-exibidor -s atributos 'tests/classes/Moderno$Ponto.class'
[4] BootstrapMethods
    Number of methods:     2
    [0]
      Bootstrap method:  cp #77 <REF_invokeStatic …/StringConcatFactory.makeConcatWithConstants : …>
      Number of arguments: 1
        Argument [0]:    cp #71 <"\u0001(\u0001, \u0001)">
    [1]
      Bootstrap method:  cp #83 <REF_invokeStatic java/lang/runtime/ObjectMethods.bootstrap : …>
      Number of arguments: 4
        Argument [0]:    cp #8 <Moderno$Ponto>
        Argument [1]:    cp #73 <"x;y">
        Argument [2]:    cp #75 <REF_getField Moderno$Ponto.x : I>
        Argument [3]:    cp #76 <REF_getField Moderno$Ponto.y : I>
```

OPÇÃO -I

# Menu interativo

- Escolhe-se a seção pelo número, quantas vezes quiser
- Entrada inválida não encerra o programa
- Para salvar tudo num arquivo: -o saida.txt



RESUMO

# O que foi e o que não foi implementado

## Implementado

- Todas as tags do pool, com índice e valor resolvido
- 202 opcodes; switches, wide e operandos do pool resolvidos
- 25 atributos, incluindo StackMapTable, anotações e Record
- Modified UTF-8, NaN/Infinity, nome da versão do Java
- Opções -s, -o e -i; erro claro para arquivo inválido
- Saída igual à do javap em 30 classes; mais de 2.000 arquivos corrompidos sem travar

## Não implementado

- Type annotations e atributos de módulo (Module, ModulePackages, ModuleMainClass): aparecem em hexadecimal
- Interface gráfica em árvore como a do jclasslib: a saída é em texto

GRUPO

# Divisão de tarefas

| Integrante | Matrícula | Parte | Arquivos |
|---|---|---|---|
| Lucas Mendes | 241020750 | Coordenação, esqueleto, parser, seções gerais, linha de comando e entrega | reader.c, classfile.c, printer.c, main.c, Makefile |
| Antonio Coelho | 241038100 | Pool de constantes | constant_pool.c |
| Gabriel Pinto | 241002331 | Bytecode | opcodes.c, bytecode.c |
| Leonardo Mileo | 241032690 | Atributos | attr_decoders.c |

Detalhes de cada membro em docs/ATIVIDADES.md. Código: github.com/LucasMABF/SB